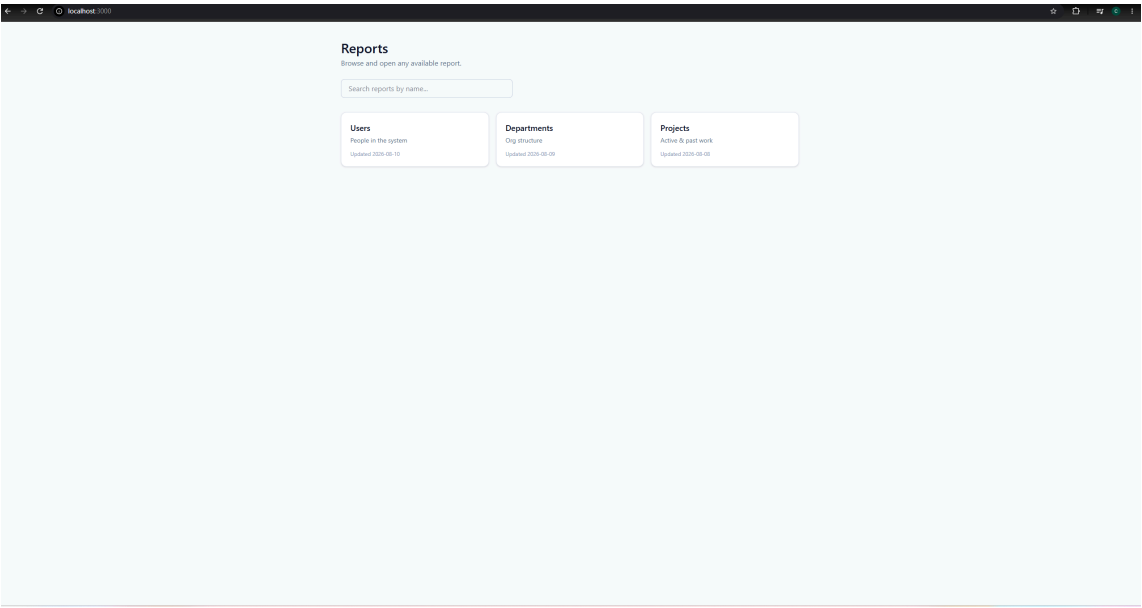


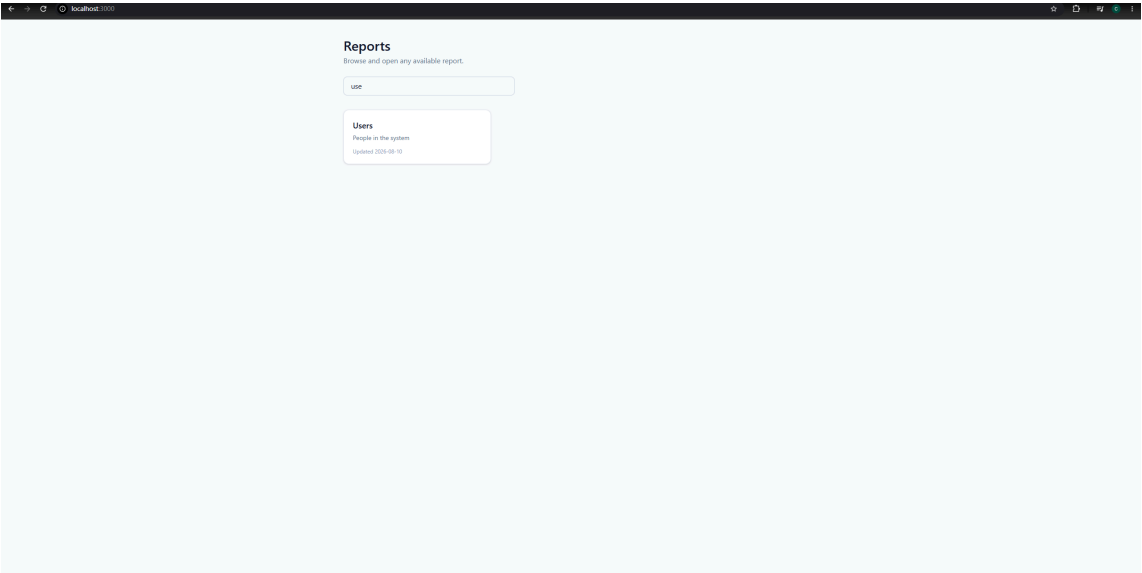
Reporting Portal — Screenshots & Assumptions/Tradeoffs

Screenshots

Landing page



Search / filter



Users report (sorted by Name)

localhost:3000/reports/users

[Back to reports](#)

Users

User ID	Name ▼	Email	Role	Status	Created Date
U015	Olivia Turner	olivia.turner@enfous.com	Editor	Active	2024-09-25
U014	Noah Becker	noah.becker@enfous.com	Viewer	Active	2024-09-02
U013	Maria Santos	maria.santos@enfous.com	Admin	Active	2024-08-06
U012	Liam O'Brien	liam.obrien@enfous.com	Editor	Inactive	2024-07-19
U011	Karin Muller	karin.muller@enfous.com	Viewer	Active	2024-07-01
U010	James Carter	james.carter@enfous.com	Editor	Active	2024-06-11
U009	Ingrid Novak	ingrid.novak@enfous.com	Admin	Active	2024-05-30
U008	Hassan Ali	hassan.ali@enfous.com	Viewer	Suspended	2024-05-09
U007	Grace Lee	grace.lee@enfous.com	Editor	Active	2024-04-22
U006	Ferdinand	ferdi.harden@enfous.com	Viewer	Active	2024-04-02
U005	Emma Wilson	emma.wilson@enfous.com	Admin	Active	2024-03-18
U004	David Kim	david.kim@enfous.com	Editor	Inactive	2024-03-05
U003	Carla Diaz	carla.diaz@enfous.com	Viewer	Active	2024-02-20
U002	Brian Chen	brian.chen@enfous.com	Editor	Active	2024-02-03
U001	Alice Johnson	alice.johnson@enfous.com	Admin	Active	2024-01-15

Departments report

localhost:3000/reports/departments

[Back to reports](#)

Departments

Department ID	Department Name	Manager	Employee Count	Location
D001	Engineering	Alice Johnson	42	San Francisco
D002	Sales	Brian Chen	28	New York
D003	Marketing	Carla Diaz	15	Austin
D004	Customer Support	David Kim	33	Remote
D005	Human Resources	Emma Wilson	9	Chicago
D006	Finance	Ferdinand	12	New York
D007	Product	Grace Lee	18	San Francisco
D008	Legal	Hassan Ali	6	Remote

Projects report

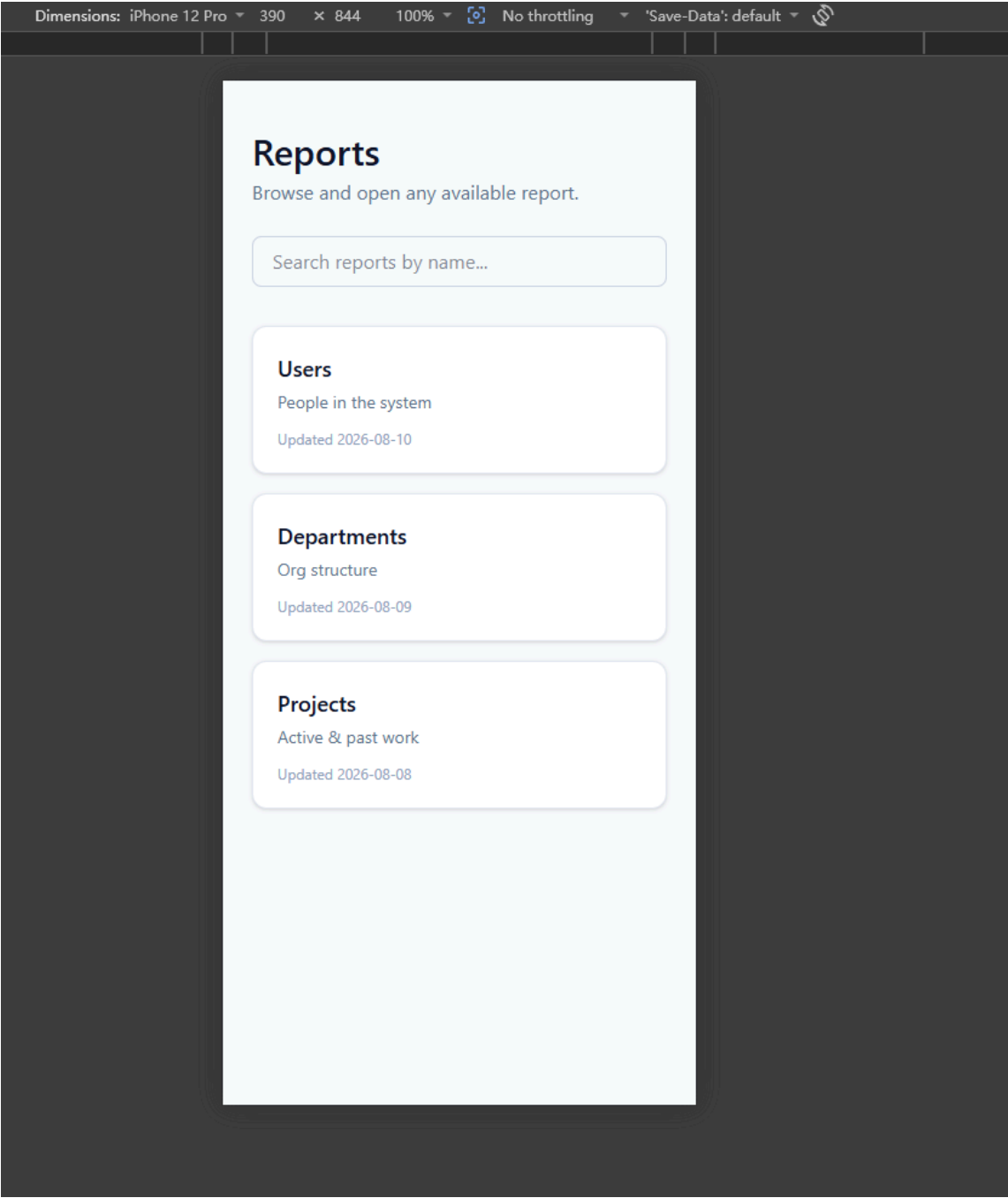
localhost:3000/projects

Back to reports

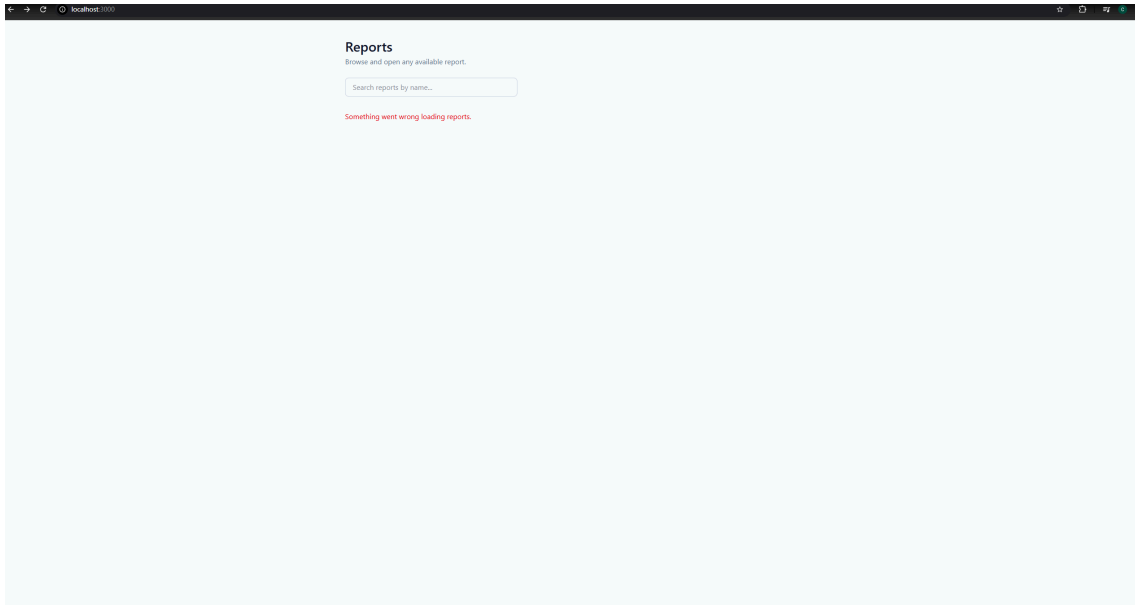
Projects

Project ID	Project Name	Department	Owner	Status	Start Date	End Date
P001	Portal Redesign	Engineering	Alice Johnson	In Progress	2024-01-06	2024-09-30
P002	Q3 Sales Campaign	Sales	Brian Chen	Completed	2024-07-01	2024-09-30
P003	Brand Refresh	Marketing	Carla Diaz	In Progress	2024-03-10	2024-09-31
P004	Support Ticket Migration	Customer Support	David Kim	Completed	2023-11-01	2024-01-15
P005	Benefits Enrollment Overhaul	Human Resources	Emma Wilson	On Hold	2024-10-05	2024-03-31
P006	Expense Automation	Finance	Farah Hashmi	In Progress	2024-02-01	2024-10-15
P007	Mobile App Beta	Product	Grace Lee	In Progress	2024-04-20	2024-11-30
P008	Vendor Contract Review	Legal	Hassan Ali	Completed	2024-08-12	2024-12-01

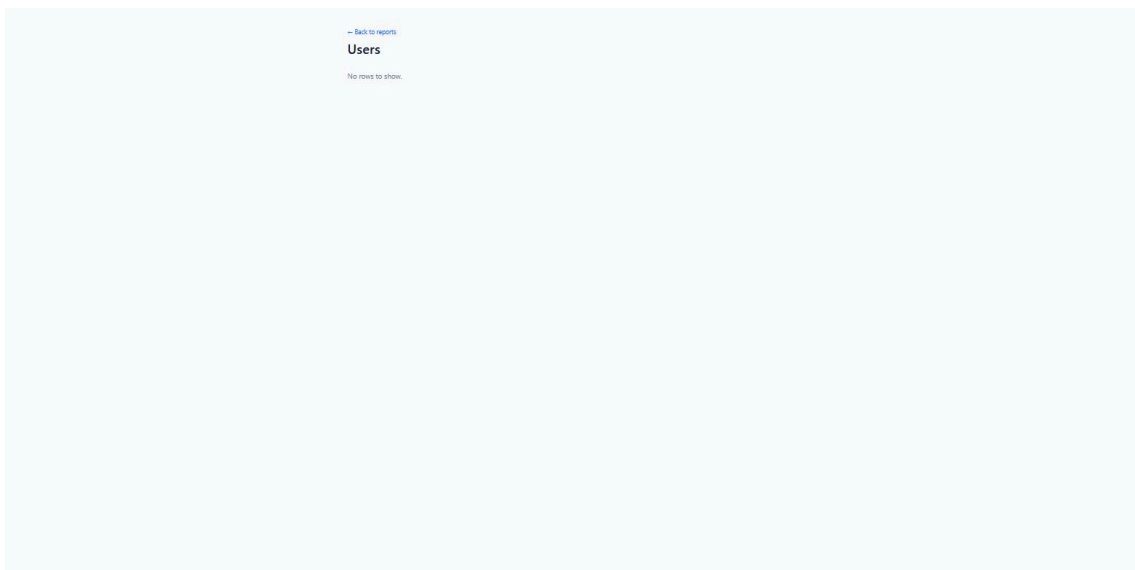
Responsive layout (mobile viewport)



Error state (backend unreachable)



Empty state (report with no rows)



Demo video: [Loom recording](#)

Assumptions and Tradeoffs

Docker Compose for one-command startup. Docker Compose provides a standardized and inspectable way to build and run the frontend and backend together with one command. The trade-off is that Docker must be installed, so the project also includes an `npm run dev` workflow for local development without Docker.

A registry-based report API instead of separate controllers. `ReportRowController` and `ReportRegistry` serve report data through `GET /api/reports/{reportId}`, including the required `/users`, `/departments`, and `/projects` routes. This avoids duplicating controller logic and makes adding another report primarily a registration task. The trade-off is that the registry introduces an additional abstraction that would be unnecessary for a single report.

Frontend-defined table columns instead of a backend-driven schema. Each report defines its columns and display order in the frontend, while the backend returns only row data. This keeps the API simple and maintains compile-time visibility of each report's presentation. The trade-off is that adding or changing a report schema may require coordinated frontend and backend changes. A metadata-driven schema could be more appropriate if reports became user-configurable.

MSW for frontend API tests. Mock Service Worker intercepts requests at the network boundary, allowing tests to exercise the application's real fetching, loading, empty, and error behavior without coupling tests to a mocked API module. This adds some test setup, but provides more realistic coverage than mocking `fetch` directly.

In-memory sample data instead of a persistence layer. The assessment does not require writes or persistence, so report data and `lastUpdated` values are static and stored in memory. This keeps the solution focused and avoids unused database, migration, and entity-mapping infrastructure. In production, I would replace this layer with persistent storage and derive update timestamps from the underlying data.